

Practice Sheet 1

Case-Study 1: E-Commerce Product Catalog

An e-commerce company is designing a product catalog that stores information about different types of products such as mobile phones, laptops, and shoes.

The company faces the following requirements:

- Different products have different attributes. For example, a mobile phone may have RAM, Storage, and Camera, while shoes may have Size, Material, and Color.
- Product attributes may change over time.
- Customers frequently search products using attributes such as brand, price, category, and specifications.
- A product may have multiple images and multiple variants.
- The system is expected to handle millions of products and a large number of concurrent users.
- Product details need to be retrieved quickly, and the system should avoid expensive joins.

Questions

- a) Which NoSQL data model would you recommend for this application: Document, Key-Value, Column-Family, or Graph? Justify your choice.
- b) Design an appropriate NoSQL representation for a mobile phone product. Show how you would represent nested attributes and multiple variants.
- c) Explain why a Document Store such as MongoDB would be more suitable than a relational database for this particular product catalog.
- d) Suppose customers frequently search using:
brand = "Samsung" AND RAM = "12GB" AND storage = "256GB"
What indexing strategy would you use?
- e) Later, the company wants to find “customers who purchased products similar to products purchased by their friends.” Would you continue using the same NoSQL data model, or would another model be more appropriate? Explain.

Case Study 2: Global E-Commerce Order System

A global e-commerce platform processes millions of orders per day across different countries. The system must remain highly available even if one or more data-center nodes fail.

Each order contains: order_id, customer_id, order_timestamp, multiple products, Product quantity and price, payment status, Shipping information, Order status history

The system has the following requirements:

- A customer should be able to retrieve all orders for a given customer within a date range.
- The system must support very high write throughput during events such as Black Friday.
- The application should continue accepting orders even if some nodes are temporarily unavailable.
- Orders should be distributed across multiple nodes.
- The system should avoid expensive joins.
- The company wants to maintain order history for several years.
- A new requirement is introduced: the system must also support queries such as “find all customers who purchased Product X in the last 30 days.”

Questions

- a) Which NoSQL data model would you select for the order system: Document, Key-Value, Column-Family, or Graph? Justify your choice based on the access patterns.
- b) Design an appropriate data model/schema for the selected NoSQL database. Clearly identify the partition key and, if applicable, clustering key.
- c) Suppose one customer has millions of orders. What problem can occur if customer_id is used directly as the partition key? How would you redesign the partitioning strategy?

- d) The application requires high availability and can tolerate temporarily stale data. Explain how you would make the system resilient to node failures using appropriate NoSQL consistency/availability choices.
- e) The new query “customers who purchased Product X in the last 30 days” does not fit naturally into the original access pattern. Would you modify the existing schema, create another denormalized table, or introduce another database? Justify your decision.
- f) Explain how your design would handle data growth, hot partitions, and high write traffic during a flash sale.

Case Study 3: Social Networking and Recommendation System

A social networking platform has 500 million users. Users can:

- Follow other users
- Create posts
- Like and comment on posts
- Follow hashtags
- Receive personalized recommendations
- Discover friends of friends
- Find users who have similar interests
- Receive a feed containing posts from users they follow

The platform has the following requirements:

- A user's profile should be retrieved quickly.
- The system must support millions of follow/unfollow operations.
- Queries such as “Who does User A follow?” should be fast.
- Queries such as “Who follows User A?” should also be fast.
- The system should efficiently answer “Find friends of friends up to two levels.”
- The system should remain available even during failures.
- The recommendation engine must perform relationship-based queries over a very large dataset.

Questions

- a) Which NoSQL data model is most appropriate for representing the relationships between users? Explain why a Graph Database may be preferable to a Document or Key-Value model.
- b) Design the basic graph model by identifying the nodes, relationships, and properties.
- c) For example, consider relationships such as:
 (User) —FOLLOWS—> (User)
 (User) —LIKED—> (Post)
 (User) —INTERESTED_IN—> (Hashtag)
 c) Consider the query:
 “Find all friends of Ram friends who are not already followed by Ram.”
 Explain why this type of multi-hop relationship traversal is easier in a Graph Database than in a traditional Document Store.
- d) The platform also needs to store the user's complete home feed containing thousands of posts. Would you store the entire feed directly inside the user's graph node? Explain your answer and propose a suitable data-storage strategy.
- e) Suppose the graph becomes extremely large and the company starts experiencing performance problems for highly connected users. What problems could arise from high-degree nodes, and what strategies could be used to address them?
- f) The company wants both:
 - relationship traversal, and
 - very high-volume post/feed storage.

Would you use one NoSQL database for everything, or adopt a polyglot persistence approach? Justify your answer with an appropriate architecture.